MOBILE APPLICATION DEVELOPMENT

AeroDiary

Assignment 1: Plan, Design and Prototype

Course	Mobile Application Development
Assignment	Assignment 1 (25 marks)
Group name	Group ALPHA
App name	AeroDiary
Date	30 October 2026

Group members

Name	Registration number
Sumama Qazi	B24S0072CS012
Muzammil Khan	B24S0876CS041
Abdullah Azam	B24S0809CS033

Pak-Austria Fachhochschule: Institute of Applied Sciences and Technology (PAF-IAST)

1. Project Idea and Problem Statement

Project statement

Most people who fly remember the trip, but forget the details that actually matter next time: which airport had decent food, whether the queues were awful, which airline's seat was comfortable, and what that great meal in transit was called. Those details end up scattered across photos, chat messages and memory, so they are lost by the time the next ticket is being booked.

AeroDiary is a simple offline journal that solves this. After a flight, the user picks two airports and a date, and the app fills in the distance and flight time by itself. The user then rates the flight, both airports and any food they tried, and can attach a photo. Over time the app builds an airport review page from the user's own ratings and suggests where to fly next based on the trips they liked most.

Target user: university students and young professionals who fly a few times a year (for study, work or visiting family) and want to remember and compare their travel experiences, rather than only tracking flights.

Why it is better than doing it manually or with a generic app: a notes app has no structure, and the big flight apps focus on tracking and itineraries. AeroDiary is built around experience: airport reviews, food and what to try next, and it works fully offline with no account.

Value proposition

For travellers who fly a few times a year and forget what each airport and flight was really like, AeroDiary is a personal travel diary app that turns their own ratings into airport reviews and smart next-trip suggestions.

Competitor review

Existing app	Strength	Weakness
Flighty	Very polished flight tracking, delay alerts and good personal flight stats.	Focused on tracking, not experiences. No airport or food reviews, and many features need a paid plan.
TripIt	Organises bookings and itineraries into one place automatically.	Built for planning, not for remembering or rating. Useful features need a paid Pro plan, and it expects an online account.

AeroDiary's gap: free, offline, no login, and centred on rating and remembering the experience, then using it to suggest the next trip.

2. Requirements

Functional requirements (12)

ID	Requirement	Priority	Built in
FR1	The user can add a journey with departure airport, arrival airport, date and airline.	Must	A2 UI, A3 logic
FR2	The user can view all journeys in a list on the Home screen and open any journey to see its full details.	Must	A2, A3
FR3	The user can edit or delete a journey (delete asks for confirmation).	Must	A2, A3
FR4	The user can search a built-in list of about 100 airports, and the system shall calculate the distance and estimated flight time automatically.	Must	A3
FR5	The user can rate a journey (seat, crew, comfort), rate the airports (cleanliness, food, queues, wifi) and log meals with a rating and note.	Must	A3
FR6	The user can attach a photo (boarding pass, food or window view) from the camera or gallery.	Must	A3
FR7	The user can search journeys by airport or airline, filter by rating or year, and sort by date.	Must	A3
FR8	The user can change settings: home airport, distance unit and the editable estimate values.	Should	A3
FR9	The user can open an Airport Reviews screen showing each airport's average ratings, built from their own entries.	Should	A4
FR10	The user can see stats: total km, countries visited, favourite airline and airport, flights per month, and badges.	Should	A4
FR11	The user can enter an optional ticket price, and the system shall show cost per km and total spend per year.	Should	A4
FR12	The system shall suggest unvisited places based on the user's highest-rated trips, and the user can save them to a wishlist.	Could	A4 if time

Could (not planned for now): comparing two airports side by side, shareable year card, route map and optional login. Login is deliberately not a Must requirement.

Non-functional requirements (4)

ID	Requirement (measurable)	How we will check it
NFR1	The Home screen loads in under 3 seconds with up to 500 saved journeys.	Test with sample data on the emulator
NFR2	All data and features work offline, with no internet connection needed.	Switch on airplane mode and test every screen
NFR3	All tappable elements are at least 48 x 48 dp.	Check in Flutter layout inspector
NFR4	All text meets a contrast ratio of at least 4.5:1 against its background.	Check colours with a contrast checker

3. Technology Stack and Justification

Part	Choice
Framework	Flutter
Language	Dart
Data storage	SQLite through the sqflite package (local, on the device)
Device integration	Camera/gallery through the image_picker package (the one integration)
Tools	Android Studio (Android emulator), VS Code, Git and GitHub

Why Flutter

1. It fits the app. AeroDiary is mostly forms, lists, cards and charts. Flutter's ready-made Material widgets cover all of that, so we spend time on features instead of styling from scratch.

2. One codebase and fast testing. Hot reload lets us change a screen and see it instantly on the emulator, which suits a team of three working across several weeks.

3. Good support for our needs. sqflite and image_picker are widely used, well documented, and have plenty of tutorials, which helps when a team member gets stuck.

Comparison with alternatives

Criteria	Flutter (chosen)	React Native	Kotlin + Jetpack Compose
Learning curve	Moderate (new language: Dart, but easy to pick up)	Moderate (JavaScript, but setup can be fiddly)	Steeper (Kotlin and Android concepts)
Performance	Very good, compiled code	Good	Best (fully native)
Community support	Large and growing	Very large	Large, Android-focused
Platform coverage	Android, iOS, web, desktop	Android and iOS	Android only
Fit for our app	Excellent for forms, lists and charts	Good	Good, but no benefit for our needs

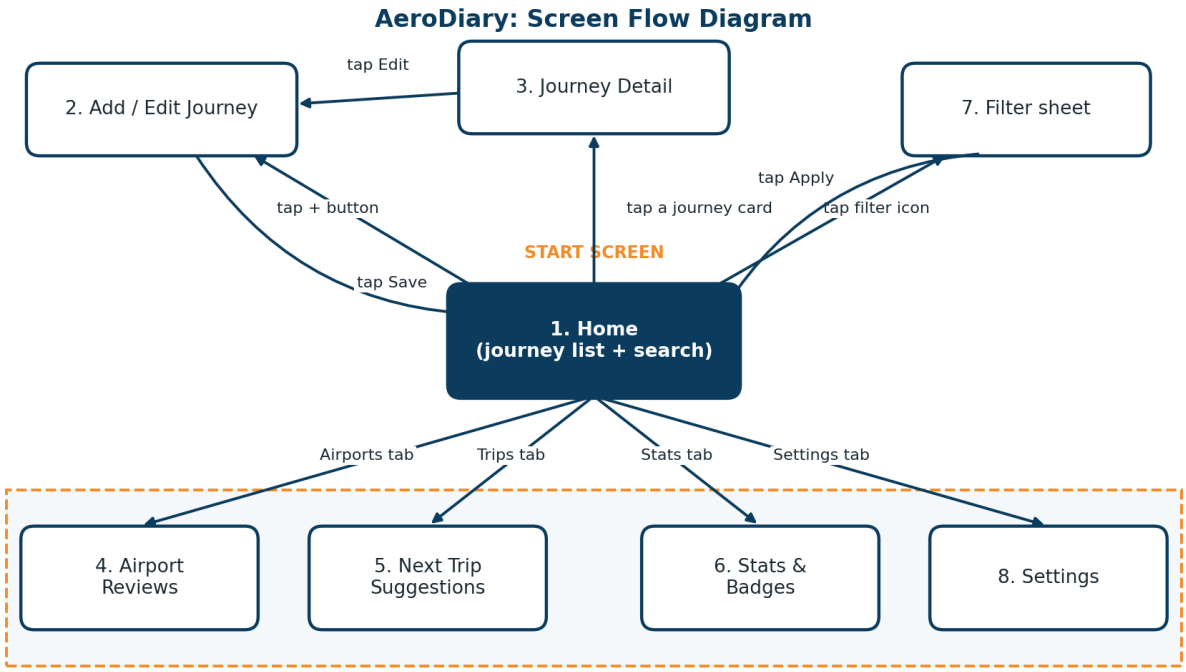
Type of approach

Flutter is a **cross-platform** framework. We write the app once in Dart and it can run on both Android and iOS. Unlike hybrid apps (which run a web page inside a wrapper), Flutter draws its own interface and compiles to native code. For us this means we build and demonstrate on Android only, as the brief allows, but could run it on iOS later without rewriting.

4. Design

The design follows a few simple user-centred ideas: one main list as the home, the most-used actions (add, search) reachable with a thumb, the same layout everywhere, and no screen more than two taps from Home.

4.1 Screen flow diagram



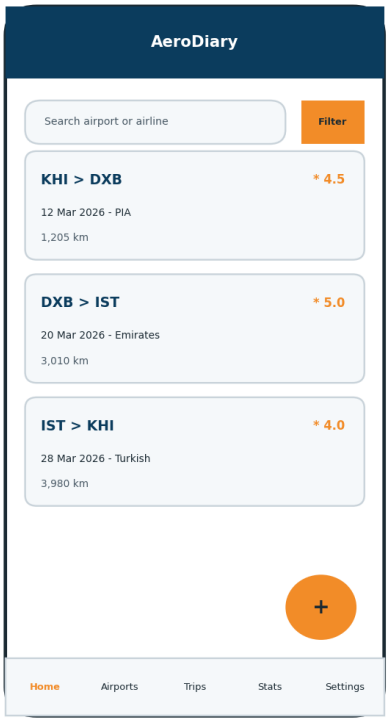
Bottom tab bar = main navigation (Home | Airports | Trips | Stats | Settings), available from every main screen

The app has **8 screens**. The start screen is Home and the main navigation is a bottom tab bar. Screens 1 to 7 are shown in the diagram and wireframes below. The filter sheet (7) opens over Home, and Settings (8) is on the last tab.

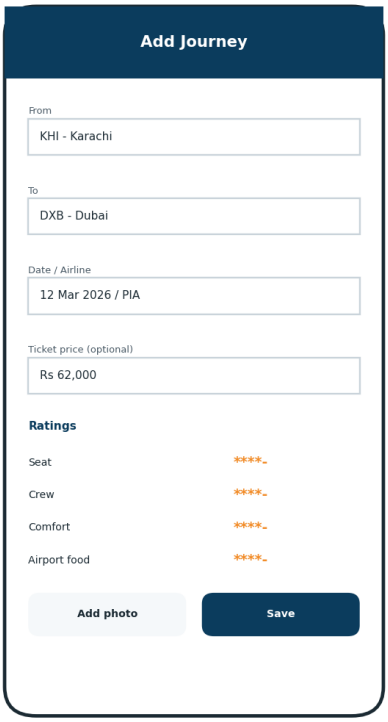
In Next Trip Suggestions, the wishlist is a tab inside the same screen, and marking a place as visited opens Add Journey with the airport pre-filled.

4.2 Wireframes

Screen 1 - Home



Screen 2 - Add / Edit Journey



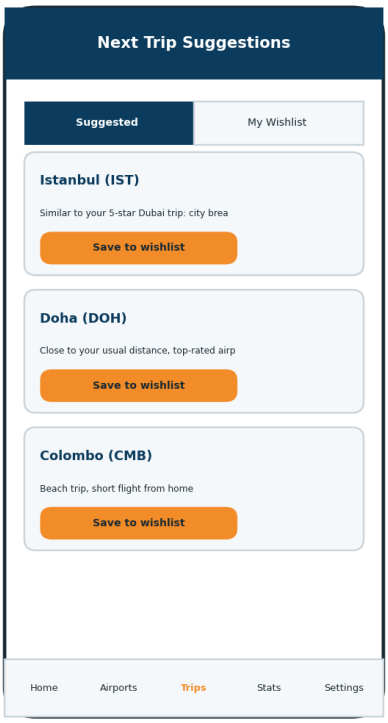
Screen 3 - Journey Detail



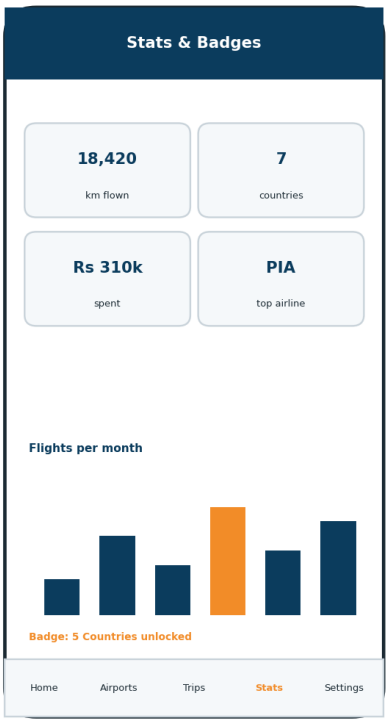
Screen 4 - Airport Reviews



Screen 5 - Next Trip Suggestions



Screen 6 - Stats & Badges



Wireframes show realistic sample content. Interactive elements are labelled on each screen (search bar, Filter button, + button, Save, Edit, Delete, Add photo, Save to wishlist, tabs and the bottom bar). The Filter sheet (7) and Settings (8) are simple lists of options and use the same components.

4.3 Style guide

Colours	Hex	Role
Navy	#0B3C5D	Primary: headers, main buttons, titles
Orange	#F28C28	Accent: ratings, add button, highlights
Light grey-blue	#F5F8FA	Background for cards and bars
Dark text	#1B2A33	All body text

Contrast check: white on navy is about 11.5:1, dark text on the light background is about 13.8:1, and dark text on orange is about 6:1. All are above the 4.5:1 target (NFR4). Orange is only used as a background or for large rating stars, never for small body text on white.

Item	Choice
Font family	Roboto (Flutter's default Material font)
Typography sizes	Headings 22 sp bold, section titles 16 sp bold, body 14 sp, captions 12 sp
Icon style	Outlined Material Icons, rounded corners, one size (24 dp) throughout
Buttons	Rounded corners (12 dp), navy for main actions, orange for add and highlight, minimum height 48 dp
Cards	Light background, thin grey border, 12 dp corner radius, 12 dp inner padding
Spacing	8 dp grid: 8 between items, 16 screen margins, 24 between sections

5. Project Setup

Item	Details
GitHub repository	**[PASTE GITHUB LINK HERE]** (the instructor is added as a collaborator, or the repo is public)
Repo structure	`aerodiary/` (Flutter app), `docs/` (report, wireframes, diagrams), `README.md`

README summary

The README contains: app name (AeroDiary), group name (ALPHA), members with IDs, project description, planned features split into Must / Should / Could (as in Section 2), the tech stack (Flutter, Dart, sqflite, image_picker) and setup instructions (install Flutter, clone the repo, run `flutter pub get` and `flutter run`).

Hello World on the emulator

[INSERT SCREENSHOT: default Flutter app running, emulator frame visible]

Commit history

[INSERT SCREENSHOT: GitHub commit history showing a commit from every member]

6. Work Plan for Assignments 2 to 4

Assignment	Task	Who
A2: UI and navigation	Build all 8 screens with dummy data, bottom tab navigation, reusable widgets (journey card, rating row, input field), apply style guide	Sumama: Home, Journey Detail, Filter Muzammil: Add/Edit Journey, Settings Abdullah: Airport Reviews, Suggestions, Stats, navigation
A3: Data and functionality	SQLite tables and CRUD, airport JSON list and distance formula, ratings and meals, camera integration, search/filter/sort, validation, settings	Sumama: database + CRUD, search/filter Muzammil: airport list + distance, ratings and meals Abdullah: camera integration, settings
A4: Testing, polish, delivery	Airport Reviews, stats, ticket price and cost, suggestions and wishlist (if time), empty states, error handling, tests, release APK, demo video, final report	Sumama: stats + cost, tests Muzammil: suggestions + wishlist, polish Abdullah: airport reviews, APK, demo video

Timeline

Weeks	Work
Week 1	A1: idea, design, repo and Hello World (this assignment)
Weeks 2 to 4	A2: all screens with dummy data
Weeks 5 to 7	A3: database, logic and camera, all Must features working
Weeks 8 to 10	A4: Should features, testing, bug fixes, APK, demo and final documents

Every Must requirement (FR1 to FR7) is built by the end of A3. Should requirements (FR8 to FR11) are finished in A3 or A4, and FR12 is only built if time allows.

Risks and how we reduce them

Risk	How we reduce it
Scope grows too big	Must features come first. Suggestions and wishlist are Could and get cut first if time runs short.
Airport data is wrong or incomplete	Use a trusted open dataset (OpenFlights) and limit it to about 100 airports. Test every distance against a known route.
A team member falls behind or is stuck	Short check-ins twice a week, and tasks can be swapped. AI tools and documentation help when someone is stuck.
Merge conflicts in Git	Each member works on their own branch and screens, and merges through pull requests reviewed by another member.
Emulator is slow or crashes	Keep a physical Android phone available for testing as a backup.

Team working agreement

Communication: a WhatsApp group for daily updates. **Meetings:** twice a week (short call or in person) plus one longer session before each deadline. **Merge conflicts:** pull before starting work, commit small and often, never push straight to the main branch, and resolve conflicts together. **Missed tasks:** tell the group early; if a task is not done by the agreed date, the others help finish it and the contribution table is updated honestly.

7. Contribution Table (Assignment 1)

Member	Tasks completed in this assignment	Contribution
Sumama Qazi	Project idea and problem statement, requirements table, report compilation and review	34%
Muzammil Khan	Wireframes, style guide, work plan and risks	33%
Abdullah Azam	GitHub repository and README, Flutter setup and Hello World, screen flow diagram	33%

8. References and Tools Used

Tool or source	Used for
Flutter documentation (docs.flutter.dev)	Framework setup, widgets and Material design
sqflite and image_picker packages (pub.dev)	Local database and camera/gallery integration
OpenFlights airport data (openflights.org/data)	Planned source for the built-in airport list
Google Fonts / Material Icons	Roboto font and icon set
GitHub and Git	Version control and repository
Android Studio	Android emulator
Claude (Anthropic AI assistant)	Brainstorming the app idea, structuring and drafting the report text and drawing the flow diagram and wireframes. All content was reviewed and agreed by the group.